Lab Assignment 3: Grid World - Policy Evaluation and Value Iteration

BL.SC.P2DSC25009
CHAMIKAR.P

Objective

Formulate a GridWorld environment as a Reinforcement Learning problem, implement Policy Evaluation and Value Iteration, and compare the paths produced by a random policy, an evaluated (given) policy, and the optimal policy.

Task 1: Formulating the Grid World Environment

The environment used for this lab is a 5-state linear Grid World (a single-row corridor), which keeps the state space small enough to fully tabulate while preserving every core RL element.

RL Component	Description
States	S1, S2, S3, S4, S5 - five discrete cells arranged in a line. The agent's state is simply which cell it currently occupies.
Actions	A = {Left (L), Right (R)}. Moving past the left edge (from S1, Left) or right edge (from S5) keeps the agent in place (boundary/wall behaviour).
Reward Structure	-1 for every action taken that does not land on the goal state; 0 for the action that transitions the agent into S5 (the goal). This encodes a shortest-path objective - every extra step costs the agent.
Terminal State(s)	S5 is the single terminal (goal) state. The episode ends as soon as the agent enters S5.
Goal of the Agent	Reach S5 from any starting state (S1 is used as the fixed start) in as few steps as possible, i.e. maximise cumulative discounted reward.

Transition dynamics: from state Si, action Right moves the agent to Si+1 (or stays at S5 if already there); action Left moves the agent to Si-1 (or stays at S1 if already there). Discount factor used: $\gamma = 0.9$.

Task 2: Policy Evaluation

Given policy evaluated: a stochastic policy that, in every non-terminal state, chooses Right with probability 0.7 and Left with probability 0.3. Policy Evaluation applies the Bellman expectation equation synchronously to every state until the value function stabilises.

Iteration	Maximum Change (Δ)	Convergence Status
1	1.00000	Not yet
2	0.90000	Not yet
3	0.81000	Not yet
4	0.55397	Not yet
5	0.45131	Not yet (Δ still decreasing; full convergence, Δ < 1e-4, is reached at iteration 46)

Resulting state values after convergence: V(S1) = -3.72, V(S2) = -3.20, V(S3) = -2.29, V(S4) = -0.87, V(S5) = 0.00 (terminal).

Observation: Δ shrinks geometrically each sweep (it is bounded by γ·Δ_prev in the worst case), which is the expected behaviour for iterative Policy Evaluation - the value estimates converge asymptotically rather than in a fixed number of steps.

Task 3: Value Iteration and Optimal Policy Generation

Value Iteration replaces the expectation over the given policy's actions with a max over actions at every sweep (the Bellman optimality backup), so it simultaneously improves the policy and evaluates it. It converged after 4 sweeps for this environment.

State	Optimal Action	State Value V*(s)
S1	Right	-2.71
S2	Right	-1.90
S3	Right	-1.00
S4	Right	0.00
S5	- (terminal)	0.00

The optimal policy is unambiguous here: always move Right, since every Right step makes strict progress toward the only reward-bearing transition (entering S5), while Left only delays arrival and accumulates more -1 penalties.

Task 4: Optimal Path Analysis

Each policy was run for one episode starting at S1 (random seed fixed for reproducibility, cap of 50 steps).

Policy Type	Path Length	Total Reward	Goal Reached (Yes/No)
Random Policy	42	-41	Yes
Evaluated Policy (70% R / 30% L)	17	-16	Yes
Optimal Policy	4	-3	Yes

Sample optimal path: S1 → S2 → S3 → S4 → S5 (4 steps, reward -3). The random and evaluated policies eventually reach the goal too, but only after many unnecessary Left moves and revisits of earlier states, which is exactly the inefficiency the optimal policy eliminates.

Analysis Questions

1. What is a state in GridWorld?

A state is a distinct configuration the agent can be in - here, one of the five cells S1-S5. It fully describes the agent's situation and is all the information the agent needs to decide its next action (the Markov property).

2. What actions can the agent perform?

Two actions are available in every state: move Left (L) or move Right (R) by one cell, with boundary cells absorbing an out-of-bounds move (e.g., Left from S1 leaves the agent at S1).

3. What is the purpose of Policy Evaluation?

Policy Evaluation computes the state-value function V^{π} for a fixed, given policy π - i.e., it answers 'how good is each state if the agent always follows this particular policy?' It does not change the policy; it only measures it, by repeatedly applying the Bellman expectation equation until V stops changing appreciably.

4. How does Value Iteration differ from Policy Evaluation?

Policy Evaluation averages over the given policy's action probabilities (an expectation), producing V^{π} for one fixed policy. Value Iteration instead takes the max over all actions at each state (the Bellman optimality backup), so each sweep implicitly improves the policy as it evaluates it. The result of Value Iteration is the optimal value function V^* and, by acting greedily on it, the optimal policy π^* - Policy Evaluation alone can never produce an improved policy on its own.

5. What is the Bellman Optimality Equation?

$V^*(s) = \max_a \Sigma_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^*(s')]$. It states that the optimal value of a state equals the expected return of the best available action, assuming optimal behaviour thereafter. In this deterministic GridWorld it simplifies to $V^*(s) = \max_a [R(s,a) + \gamma V^*(s')]$, which is exactly the backup used in Task 3.

6. Which policy reached the goal using the fewest steps?

The Optimal Policy, reaching S5 in 4 steps (S1→S2→S3→S4→S5), versus 17 steps for the evaluated policy and 42 steps for the purely random policy.

7. Why is the optimal policy superior to a random policy?

The optimal policy is derived from V*, so at every state it picks the action that maximises expected discounted return rather than choosing arbitrarily. Concretely this means it never wastes a step moving away from the goal, so it accumulates far less negative reward (-3 vs -41) and reaches the terminal state in the minimum possible number of steps, whereas a random policy has no mechanism to avoid backtracking.

8. How would obstacles affect the optimal path?

Obstacles (blocked or high-penalty cells) would remove or heavily penalise certain transitions, so Value Iteration would back up lower values for states adjacent to an obstacle and the greedy policy would route around it - the optimal path would no longer be the straight-line shortest path but the shortest path that avoids the obstacle. If an obstacle fully disconnects the goal from a state, V*(s) for that state would reflect the reward of the best remaining (possibly much longer, or non-existent) route, and the corresponding optimal action would point along whatever detour minimises the penalty.

Conclusion

Policy Evaluation gave a way to quantify how good a specific, fixed policy is: starting from V = 0 for every state, repeated Bellman-expectation backups drove the value estimates down to their true values under the 70/30 policy (e.g., V(S1) converging to about -3.72), with the maximum change per sweep shrinking geometrically toward zero rather than stopping abruptly.

Value Iteration went a step further by replacing the policy-weighted average with a max over actions at every backup, so it discovered the optimal value function and, by acting greedily with respect to it, the optimal policy (always move Right) in only 4 sweeps - far fewer than full evaluation of the sub-optimal policy required.

The random policy has no notion of progress and can wander for many steps (42 in this run, reward -41) before stumbling onto the goal; the evaluated 70/30 policy is biased toward the goal and does better (17 steps, reward -16) but still wastes moves on the 30% of the time it steps Left; the optimal policy is fully directed and reaches the goal in the minimum 4 steps with reward -3.

Key observation from the shortest-path comparison: the total reward obtained is directly tied to path length in this environment (reward = -(steps to goal)), so minimising steps and maximising cumulative reward are the same objective here - which is exactly why Value Iteration's greedy, value-maximising policy also turns out to be the shortest-path policy.